

Pseudocodes

Part-1

1. What will be the output of the following pseudocode for A = [4, 8, 15, 16, 23], n = 5?

```
void rearrange(Integer A[], Integer n)
    Integer i = 0, j = n - 1
    while (i < j)
        swap(A[i], A[i+1])
        i = i + 2
        j = j - 1
    end while
    for k = 0 to n-1
        Print A[k]
    end for
end function
```

- a. 8 4 16 15 23
- b. 4 8 15 16 23
- c. 8 4 15 16 23
- d. 4 8 16 15 23

Answer: 8 4 16 15 23



Explanation:

Initial: A = [4, 8, 15, 16, 23], i = 0, j = 4
Check while (i < j) → 0 < 4 → true
swap(A[0], A[1]) → swap 4 and 8 → A = [8, 4, 15, 16, 23]
i = i + 2 → i = 2; j = j - 1 → j = 3
Check while (i < j) → 2 < 3 → true
swap(A[2], A[3]) → swap 15 and 16 → A = [8, 4, 16, 15, 23]
i = i + 2 → i = 4; j = j - 1 → j = 2
Check while (i < j) → 4 < 2 → false → loop ends
Print array from k=0 to 4 → 8 4 16 15 23

2. What will be the output of the following pseudocode for S = "ACCENTURE"?

```
void func(String S)
    Integer count = 0
    for i = 0 to length(S) - 1
        if (S[i] == 'A' or S[i] == 'E' or S[i] == 'I' or S[i] == 'O' or S[i] == 'U')
            count = count + 1
```

```
    end if
  end for
  Print count
end function
```

- a. 2
- b. 3
- c. 4
- d. 5

Answer: 4

Explanation:

S = "A C C E N T U R E", indices 0 to 8
i=0: S[0]='A' → vowel → count=1
i=1: S[1]='C' → not vowel → count=1
i=2: S[2]='C' → not vowel → count=1
i=3: S[3]='E' → vowel → count=2
i=4: S[4]='N' → not vowel → count=2
i=5: S[5]='T' → not vowel → count=2
i=6: S[6]='U' → vowel → count=3
i=7: S[7]='R' → not vowel → count=3
i=8: S[8]='E' → vowel → count=4
Print count → 4



3. What will be the output of the following pseudocode for x = 5, y = 3?

```
Integer compute(Integer x, Integer y)
. Integer result = (x << 1) ^ y
. return result
. end function
```

- a. 8
- b. 11
- c. 9
- d. 13

Answer: 9

Explanation:

x = 5 → binary: 0101
x << 1 (left shift by 1) → 1010 → decimal 10
y = 3 → binary: 0011
result = 10 ^ 3 → XOR of 1010 and 0011
Bit by bit XOR: 1010 XOR 0011 = 1001

```
1001 in decimal = 9  
return result → 9
```

4. What will be the output of the following pseudocode?

```
void stackOps()  
    Stack S = empty  
    push(S, 10)  
    push(S, 20)  
    push(S, 30)  
    pop(S)  
    push(S, 40)  
    Print top(S)  
    Print size(S)  
end function
```

- a. 30,3
- b. 40,2
- c. 30,2
- d. 40,3

Answer: 40,3

Explanation:

```
push(S,10) → Stack: [10] (bottom to top)  
push(S,20) → Stack: [10, 20]  
push(S,30) → Stack: [10, 20, 30]  
pop(S) → removes top element (30) → Stack: [10, 20]  
push(S,40) → Stack: [10, 20, 40]  
top(S) → returns 40 (topmost element)  
size(S) → returns 3 (total elements: 10, 20, 40)  
Output: 40, 3
```



5. What will be the output of the following pseudocode?

```
void queueOps()  
    Queue Q = empty  
    enqueue(Q, 5)  
    enqueue(Q, 10)  
    dequeue(Q)  
    enqueue(Q, 15)  
    enqueue(Q, 20)  
    dequeue(Q)  
    Print front(Q)  
    Print size(Q)  
end function
```

- a. 10,3
- b. 15,3
- c. 10,2
- d. 15,2

Answer: 15,2

Explanation:

```
enqueue(Q,5) → Queue (front→rear): [5]
enqueue(Q,10) → Queue: [5, 10]
dequeue(Q) → removes front element (5) → Queue: [10]
enqueue(Q,15) → Queue: [10, 15]
enqueue(Q,20) → Queue: [10, 15, 20]
dequeue(Q) → removes front element (10) → Queue: [15, 20]
front(Q) → returns 15 (frontmost element)
size(Q) → returns 2 (total elements: 15, 20)
Output: 15, 2
```

6. What will be the output of the following pseudocode?

```
Integer fib(Integer n)
  if (n <= 1)
    return n
  else
    return fib(n-1) + fib(n-2)
  end if
end function
Print fib(6)
```



- a. 5
- b. 8
- c. 13
- d. 6

Answer: 8

Explanation:

```
fib(0) = 0, fib(1) = 1 (base cases)
fib(2) = fib(1) + fib(0) = 1 + 0 = 1
fib(3) = fib(2) + fib(1) = 1 + 1 = 2
fib(4) = fib(3) + fib(2) = 2 + 1 = 3
fib(5) = fib(4) + fib(3) = 3 + 2 = 5
fib(6) = fib(5) + fib(4) = 5 + 3 = 8
Print fib(6) → 8
```

7. What will be the output of the following pseudocode for array $A = [_, 1, 2, 3, 4, 5, 6, 7]$ (1-indexed, index 0 unused), $n = 7$?

(Left child of node $i = 2i$, Right child $= 2i+1$; a node is a leaf if $2i > n$)

```
Integer func(Integer A[], Integer i, Integer n)
    if (2*i > n)
        return A[i]
    end if
    Integer sum = 0
    if (2*i <= n)
        sum = sum + func(A, 2*i, n)
    end if
    if (2*i+1 <= n)
        sum = sum + func(A, 2*i+1, n)
    end if
    return sum
end function
Print func(A, 1, 7)
```

a. 18

b. 28

c. 22

d. 15

Answer: 22

Explanation:



Tree structure (1-indexed heap array): node 1 has children 2,3; node 2 has children 4,5; node 3 has children 6,7

Since $n = 7$, nodes 4, 5, 6, 7 have $2i > 7 \rightarrow$ they are leaves

```
func(4):  $2*4 = 8 > 7 \rightarrow$  leaf  $\rightarrow$  return  $A[4] = 4$ 

func(5):  $2*5 = 10 > 7 \rightarrow$  leaf  $\rightarrow$  return  $A[5] = 5$ 

func(6):  $2*6 = 12 > 7 \rightarrow$  leaf  $\rightarrow$  return  $A[6] = 6$ 

func(7):  $2*7 = 14 > 7 \rightarrow$  leaf  $\rightarrow$  return  $A[7] = 7$ 

func(2) = func(4) + func(5)
        = 4 + 5
        = 9

func(3) = func(6) + func(7)
        = 6 + 7
        = 13

func(1) = func(2) + func(3)
        = 9 + 13
        = 22
```

8. What will be the output of the following pseudocode for the adjacency matrix below (n = 4)?

```
A =  
0 1 1 0  
1 0 1 1  
1 1 0 0  
0 1 0 0
```

```
Integer funn(Integer A[], Integer n)  
    Integer count = 0  
    for i = 0 to n-1  
        for j = i+1 to n-1  
            if (A[i][j] == 1)  
                count = count + 1  
            end if  
        end for  
    end for  
    return count  
end function  
Print funn(A, 4)
```

- a. 5
- b. 6
- c. 4
- d. 3

Answer: 4

Explanation:



Since the matrix is symmetric (undirected graph), only check pairs where $j > i$ to avoid double counting
i=0: j=1 $\rightarrow A[0][1]=1 \rightarrow \text{count}=1$; j=2 $\rightarrow A[0][2]=1 \rightarrow \text{count}=2$; j=3 $\rightarrow A[0][3]=0 \rightarrow$ no change
i=1: j=2 $\rightarrow A[1][2]=1 \rightarrow \text{count}=3$; j=3 $\rightarrow A[1][3]=1 \rightarrow \text{count}=4$
i=2: j=3 $\rightarrow A[2][3]=0 \rightarrow$ no change
i=3: no $j > 3$ within range
Final count = 4
Print funn(A,4) $\rightarrow 4$

9. What will be the output of the following pseudocode for $A = [5, 3, 8, 4, 2]$, $n = 5$ (single pass only)?

```
void onePass(Integer A[], Integer n)  
    for i = 0 to n-2  
        if (A[i] > A[i+1])  
            swap(A[i], A[i+1])  
        end if  
    end for  
    Print A  
end function
```

- a. 3 5 4 2 8

b. 2 3 4 5 8

c. 3 4 5 2 8

d. 3 5 8 4 2

Answer: 3 5 4 2 8

Explanation:

```
Initial: A = [5, 3, 8, 4, 2]
i=0: compare A[0]=5, A[1]=3 → 5>3 → swap → A = [3, 5, 8, 4, 2]
i=1: compare A[1]=5, A[2]=8 → 5>8 is false → no swap → A = [3, 5, 8, 4, 2]
i=2: compare A[2]=8, A[3]=4 → 8>4 → swap → A = [3, 5, 4, 8, 2]
i=3: compare A[3]=8, A[4]=2 → 8>2 → swap → A = [3, 5, 4, 2, 8]
Loop ends (i goes only up to n-2 = 3)
Print A → 3 5 4 2 8
```

10. What will be the index (0-based) returned by the following pseudocode for A = [2, 4, 6, 8, 10, 12, 14], target = 10?

```
Integer funn(Integer A[], Integer n, Integer target)
    Integer low = 0, high = n-1
    while (low <= high)
        Integer mid = (low + high) / 2
        if (A[mid] == target)
            return mid
        else if (A[mid] < target)
            low = mid + 1
        else
            high = mid - 1
        end if
    end while
    return -1
end function
Print funn(A, 7, 10)
```

a. -1

b. 5

c. 4

d. 3

Answer: 4

Explanation:

```
low=0, high=6
mid = (0+6)/2 = 3 → A[3] = 8 → 8 < 10 → low = mid+1 = 4
low=4, high=6 → mid = (4+6)/2 = 5 → A[5] = 12 → 12 > 10 → high = mid-1 = 4
```

low=4, high=4 $\rightarrow$ mid = (4+4)/2 = 4 $\rightarrow$ A[4] = 10 $\rightarrow$ matches target $\rightarrow$ return 4
Print funn(A,7,10) $\rightarrow$ 4

11. What will be the output of the following pseudocode?

```
Integer a = 5
Integer b = (a++) + (++a)
Print a
Print b
```

a. 6 12

b. 7 12

c. 7 13

d. 6 13

Answer: 7 12

Explanation:

```
a = 5 initially
a++ (post-increment)  $\rightarrow$  uses current value of a (5) in the expression, THEN increments
a  $\rightarrow$  a becomes 6
++a (pre-increment)  $\rightarrow$  increments a first  $\rightarrow$  a becomes 7, THEN uses updated value (7)
in the expression
b = 5 (from a++) + 7 (from ++a) = 12
Final value of a = 7
Print a  $\rightarrow$  7
Print b  $\rightarrow$  12
```

12. What will be the output of the following pseudocode for matrix A (3x3)?

```
A =
1 2 3
4 5 6
7 8 9
```

```
Integer upperSum(Integer A[][], Integer n)
    Integer sum = 0
    for i = 0 to n-1
        for j = 0 to n-1
            if (i < j)
                sum = sum + A[i][j]
            end if
        end for
    end for
    return sum
end function
Print upperSum(A, 3)
```

- a. 6
- b. 24
- c. 11
- d. 15

Answer: 11

Explanation:

Condition $i < j$ only picks elements strictly above the main diagonal
 $i=0: j=0 \rightarrow 0 < 0$ false; $j=1 \rightarrow 0 < 1$ true $\rightarrow$ sum $+= A[0][1]=2 \rightarrow$ sum=2; $j=2 \rightarrow 0 < 2$ true $\rightarrow$ sum $+= A[0][2]=3 \rightarrow$ sum=5
 $i=1: j=0 \rightarrow$ false; $j=1 \rightarrow$ false; $j=2 \rightarrow 1 < 2$ true $\rightarrow$ sum $+= A[1][2]=6 \rightarrow$ sum=11
 $i=2: j=0,1,2 \rightarrow$ all $i < j$ false (since $i=2$ is largest)
 Final sum = 11
 Print upperSum(A,3) $\rightarrow$ 11

13. What will be the output of the following pseudocode for $S = \text{"ABC"}$?

```
void shiftChars(String S)
  for i = 0 to length(S) - 1
    Integer code = ascii(S[i]) + 2
    Print char(code)
  end for
end function
```



- a. CDE
- b. BCD
- c. DEF
- d. CDA

Answer: CDE

Explanation:

$S[0] = \text{'A'} \rightarrow \text{ascii}(\text{'A'}) = 65 \rightarrow 65+2 = 67 \rightarrow \text{char}(67) = \text{'C'}$
 $S[1] = \text{'B'} \rightarrow \text{ascii}(\text{'B'}) = 66 \rightarrow 66+2 = 68 \rightarrow \text{char}(68) = \text{'D'}$
 $S[2] = \text{'C'} \rightarrow \text{ascii}(\text{'C'}) = 67 \rightarrow 67+2 = 69 \rightarrow \text{char}(69) = \text{'E'}$
 Printed one after another $\rightarrow$ C, D, E
 Final output $\rightarrow$ CDE

14. What will be the output of the following pseudocode for $n = 13$?

```
Integer countSetBits(Integer n)
    Integer count = 0
    while (n > 0)
        count = count + (n mod 2)
        n = n / 2
    end while
    return count
end function
Print countSetBits(13)
```

- a. 5
- b. 4
- c. 2
- d. 3

Answer: 3

Explanation:

```
n = 13 → binary = 1101
Iteration 1: n=13 → 13 mod 2 = 1 → count=1; n = 13/2 = 6 (integer division)
Iteration 2: n=6 → 6 mod 2 = 0 → count=1; n = 6/2 = 3
Iteration 3: n=3 → 3 mod 2 = 1 → count=2; n = 3/2 = 1
Iteration 4: n=1 → 1 mod 2 = 1 → count=3; n = 1/2 = 0
n=0 → loop ends
Print countSetBits(13) → 3
```

15. What will be the output of the following pseudocode for marks = 82?

```
Character grade(Integer marks)
    if (marks >= 90)
        return 'A'
    else if (marks >= 75)
        return 'B'
    else if (marks >= 50)
        return 'C'
    else
        return 'D'
    end if
end function
Print grade(82)
```

- a. A
- b. B

c. C

d. D

Answer: B

Explanation:

```
marks = 82
Check marks >= 90 → 82 >= 90 → false → move to next condition
Check marks >= 75 → 82 >= 75 → true → return 'B' immediately (rest of the chain is
skipped)
Function returns 'B'
Print grade(82) → B
```

16. What will be the output of the following pseudocode for list $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$?

```
void funnn(Node head)

    Node prev = NULL

    Node curr = head

    while (curr != NULL)

        Node next = curr.next

        curr.next = prev

        prev = curr

        curr = next

    end while

    printList(prev)

end function
```



a. 10 20 30 40

b. 40 30 20 10 NULL

c. Error

d. 40 30 20 10

Answer: 40 30 20 10

Explanation:

Initial: prev=NULL, curr=10→20→30→40
Iter1: next=20; curr(10).next=prev(NULL) → 10→NULL; prev=10; curr=20
Iter2: next=30; curr(20).next=prev(10) → 20→10→NULL; prev=20; curr=30
Iter3: next=40; curr(30).next=prev(20) → 30→20→10→NULL; prev=30; curr=40
Iter4: next=NULL; curr(40).next=prev(30) → 40→30→20→10→NULL; prev=40; curr=NULL
Loop ends (curr==NULL); printList(prev) starts at 40
Output: 40 30 20 10

17. What will be the output of the following pseudocode for A = [1, 2, 3, 2, 1, 4, 1]?

```
void func(Integer A[], Integer n)

    Map freq = empty

    for i = 0 to n-1

        if (freq.contains(A[i]))

            freq[A[i]] = freq[A[i]] + 1

        else

            freq[A[i]] = 1

        end if

    end for

    Print freq[1]

end function

func(A, 7)
```

- a. 3
- b. 4
- c. 1
- d. 2

Answer: 3

Explanation:

A = [1, 2, 3, 2, 1, 4, 1]

i=0: A[0]=1 → not in map → freq[1]=1

i=1: A[1]=2 → not in map → freq[2]=1

i=2: A[2]=3 → not in map → freq[3]=1

i=3: A[3]=2 → in map → freq[2]=2
i=4: A[4]=1 → in map → freq[1]=2
i=5: A[5]=4 → not in map → freq[4]=1
i=6: A[6]=1 → in map → freq[1]=3
Print freq[1] → 3

18. What will be the output of the following pseudocode for n = 5?

```
Integer countWays(Integer n)
    Integer dp[n+1]
    dp[0] = 1
    dp[1] = 1
    for i = 2 to n
        dp[i] = dp[i-1] + dp[i-2]
    end for
    return dp[n]
end function
Print countWays(5)
```

- a. 5
- b. 13
- c. 8
- d. 9



Answer: 8

Explanation:

```
dp[0]=1, dp[1]=1
dp[2] = dp[1] + dp[0] = 1 + 1 = 2
dp[3] = dp[2] + dp[1] = 2 + 1 = 3
dp[4] = dp[3] + dp[2] = 3 + 2 = 5
dp[5] = dp[4] + dp[3] = 5 + 3 = 8
return dp[5] → 8
Print countWays(5) → 8
```

19. What will be the output of the following pseudocode?

```
void pattern()
    Integer count = 0
    for i = 1 to 3
        for j = 1 to i
            count = count + j
```

```

        end for
    end for
    Print count
end function

```

- a. 9
- b. 14
- c. 6
- d. 10

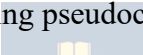
Answer: 10

Explanation:

```

i=1: j=1 → count += 1 → count=1
i=2: j=1 → count += 1 → count=2; j=2 → count += 2 → count=4
i=3: j=1 → count += 1 → count=5; j=2 → count += 2 → count=7; j=3 → count += 3 →
count=10
Final count = 10
Print count → 10

```

20. What will be the output of the following pseudocode for postfix expression "5 3 2 * +"? 

```

Integer funn(String expr)
    Stack S = empty
    for each token in expr (space separated)
        if (token is a number)
            push(S, token)
        else
            Integer b = pop(S)
            Integer a = pop(S)
            if (token == '*')
                push(S, a * b)
            else if (token == '+')
                push(S, a + b)
            end if
        end if
    end for
    return pop(S)
end function
Print funn("5 3 2 * +")

```

- a. 11
- b. 16
- c. 21
- d. 13

Answer: 11

Explanation:

```

token "5" → number → push → Stack: [5]
token "3" → number → push → Stack: [5, 3]
token "2" → number → push → Stack: [5, 3, 2]
token "" → pop b=2, pop a=3 → push ab = 3*2=6 → Stack: [5, 6]
token "+" → pop b=6, pop a=5 → push a+b = 5+6=11 → Stack: [11]
return pop(S) → 11
Print funn(...) → 11

```

21. What will be the output of the following pseudocode for a circular queue of capacity 3?

```

void circularOps()
    CircularQueue Q(capacity=3)
    enqueue(Q, 1)
    enqueue(Q, 2)
    enqueue(Q, 3)
    dequeue(Q)
    enqueue(Q, 4)
    Print front(Q)
    Print rear(Q)
end function

```

- a. 1,4
- b. 2,4
- c. 2,3
- d. 1,3



Answer: 2,4

Explanation:

```

enqueue(1) → Queue: [1] (front=1, rear=1)
enqueue(2) → Queue: [1,2] (front=1, rear=2)
enqueue(3) → Queue: [1,2,3] (front=1, rear=3) — queue is now full (capacity 3)
dequeue() → removes front (1) → Queue: [2,3] (front=2, rear=3)
enqueue(4) → space freed by dequeue → Queue: [2,3,4] (front=2, rear=4, wrapped circularly)
front(Q) → 2
rear(Q) → 4
Output: 2, 4

```

22. What will be the BFS traversal output starting from node 0 for the following adjacency list?

- 0: [1, 2]
- 1: [0, 3]
- 2: [0, 3]

3: [1, 2]

```
void funn(Integer start)
    Queue Q = empty
    Set visited = empty
    enqueue(Q, start)
    visited.add(start)
    while (Q is not empty)
        Integer node = dequeue(Q)
        Print node
        for each neighbor in adjList[node]
            if (neighbor not in visited)
                visited.add(neighbor)
                enqueue(Q, neighbor)
            end if
        end for
    end while
end function
funn(0)
```

a. 0 3 1 2

b. 0 1 2 3

c. 0 2 1 3

d. 0 1 3 2

Answer: 0 1 2 3



Explanation:

Start: enqueue(0), visited={0} → Queue: [0]
Dequeue 0 → Print 0 → Queue: []; neighbors of 0: [1,2] → both unvisited → enqueue both
→ visited={0,1,2} → Queue: [1,2]
Dequeue 1 → Print 1 → Queue: [2]; neighbors of 1: [0,3] → 0 visited, 3 unvisited →
enqueue 3 → visited={0,1,2,3} → Queue: [2,3]
Dequeue 2 → Print 2 → Queue: [3]; neighbors of 2: [0,3] → both visited → no changes
Dequeue 3 → Print 3 → Queue: []; neighbors of 3: [1,2] → both visited → no changes
Output order: 0 1 2 3

23. What will be the output of the following pseudocode?

```
Integer mystery(Integer n)
    if (n == 0)
        return 0
    end if
    return n + mystery(n - 1)
end function
Print mystery(4)
```

a. 10

b. 8

- c. 6
- d. 24

Answer: 10

Explanation:

```
mystery(4) = 4 + mystery(3)
mystery(3) = 3 + mystery(2)
mystery(2) = 2 + mystery(1)
mystery(1) = 1 + mystery(0)
mystery(0) = 0 (base case)
Unwind: mystery(1) = 1+0 = 1
mystery(2) = 2+1 = 3
mystery(3) = 3+3 = 6
mystery(4) = 4+6 = 10
Print mystery(4) → 10
```

24. What will be the output of the following pseudocode for A = [1, 2, 3, 4, 5], k = 2?

```
void funn(Integer A[], Integer n, Integer k)
    for i = 0 to k-1
        Integer temp = A[0]
        for j = 0 to n-2
            A[j] = A[j+1]
        end for
        A[n-1] = temp
    end for
    Print A
end function
funn(A, 5, 2)
```

- a. 1 2 3 4 5
- b. 4 5 1 2 3
- c. 2 3 4 5 1
- d. 3 4 5 1 2

Answer: 3 4 5 1 2

Explanation:

```
Initial: A = [1,2,3,4,5]
Outer i=0 (1st rotation): temp=A[0]=1; shift left: A=[2,3,4,5,5]; A[4]=temp=1 → A=
[2,3,4,5,1]
Outer i=1 (2nd rotation): temp=A[0]=2; shift left: A=[3,4,5,1,1]; A[4]=temp=2 → A=
[3,4,5,1,2]
Loop ends (k=2 rotations done)
Print A → 3 4 5 1 2
```

25. What will be the output of the following pseudocode for S = "MALAYALAM"?

```
Boolean funn(String S)
    Integer left = 0, right = length(S) - 1
    while (left < right)
        if (S[left] != S[right])
            return false
        end if
        left = left + 1
        right = right - 1
    end while
    return true
end function
Print funn("MALAYALAM")
```

- a. true
- b. false
- c. error
- d. 1

Answer: true

Explanation:

```
S = "MALAYALAM" (length 9), left=0, right=8
S[0]='M', S[8]='M' → match → left=1, right=7
S[1]='A', S[7]='A' → match → left=2, right=6
S[2]='L', S[6]='L' → match → left=3, right=5
S[3]='A', S[5]='A' → match → left=4, right=4
left < right fails (4 < 4 false) → loop ends
return true → isPalindrome → true
```

26. What will be the output of the following pseudocode for a = 15, b = 10?

```
Integer a = 15
Integer b = 10
Integer result = (a > b) ? (a - b) : (b - a)
Print result
```

- a. 10
- b. 25
- c. 5
- d. -5

Answer: 5

Explanation:

Evaluate condition: $a > b \rightarrow 15 > 10 \rightarrow \text{true}$
Since condition is true, ternary returns first expression: $(a - b)$
 $a - b = 15 - 10 = 5$
result = 5
Print result $\rightarrow 5$

27. What will be the output of the following pseudocode for $A = [-2, 3, -1, 4, -2, 5]$, $n = 6$?

```
Integer maxSubSum(Integer A[], Integer n)
    Integer maxSum = A[0], currSum = A[0]
    for i = 1 to n-1
        currSum = max(A[i], currSum + A[i])
        maxSum = max(maxSum, currSum)
    end for
    return maxSum
end function
Print maxSubSum(A, 6)
```

- a. 9
- b. 7
- c. 5
- d. 11

Answer: 9



Explanation:

```
maxSum=-2, currSum=-2
i=1: A[1]=3  $\rightarrow$  currSum=max(3, -2+3=1)=3  $\rightarrow$  maxSum=max(-2, 3)=3
i=2: A[2]=-1  $\rightarrow$  currSum=max(-1, 3-1=2)=2  $\rightarrow$  maxSum=max(3, 2)=3
i=3: A[3]=4  $\rightarrow$  currSum=max(4, 2+4=6)=6  $\rightarrow$  maxSum=max(3, 6)=6
i=4: A[4]=-2  $\rightarrow$  currSum=max(-2, 6-2=4)=4  $\rightarrow$  maxSum=max(6, 4)=6
i=5: A[5]=5  $\rightarrow$  currSum=max(5, 4+5=9)=9  $\rightarrow$  maxSum=max(6, 9)=9
return maxSum  $\rightarrow 9$ 
```

28. What will be the output of the following pseudocode?

```
void modify(Integer x)
    x = x + 10
end function

Integer a = 5
modify(a)
Print a
```

- a. 15

- b. 5
- c. 10
- d. error

Answer: 5

Explanation:

a = 5 (in the calling scope)
modify(a) is called → since parameters are passed by value, a copy of a's value (5) is passed to x
Inside modify: $x = x + 10$ → x becomes 15, but this only changes the local copy x, not the original a
Function returns without affecting a
Print a → 5 (unchanged)

29. What will be the output of the following pseudocode?

```
void findFirst(Integer A[], Integer n)
    Integer i = 0
    Integer result = -1
    while (i < n and result == -1)
        if (A[i] mod 7 == 0)
            result = A[i]
        end if
        i = i + 1
    end while
    Print result
end function
findFirst([3, 8, 14, 21, 28], 5)
```

- a. 14
- b. 21
- c. 28
- d. 25

Answer: 14

Explanation:

i=0: $A[0]=3 \rightarrow 3 \bmod 7 = 3 \neq 0 \rightarrow$ no change; i=1
i=1: $A[1]=8 \rightarrow 8 \bmod 7 = 1 \neq 0 \rightarrow$ no change; i=2
i=2: $A[2]=14 \rightarrow 14 \bmod 7 = 0 \rightarrow$ result=14; i=3
Check while condition: result == -1 is now false → loop ends
Print result → 14

30. What will be the output of the following pseudocode for $n = 16$?

```
Boolean funn(Integer n)
  if (n <= 0)
    return false
  end if
  return (n & (n - 1)) == 0
end function
Print funn(16)
```

- a. true
- b. false
- c. 0
- d. Error

Answer: true

Explanation:

```
n = 16 → binary: 10000
n - 1 = 15 → binary: 01111
n & (n-1) → 10000 AND 01111 = 00000 (bit-by-bit AND, no overlapping 1 bits)
Result = 0 → condition (result == 0) is true
return true → funn(16) → true (since 16 = 2^4, it is indeed a power of 2)
```

KN ACADEMY
E-LEARNING

Part-2

31. What will be the output of the following pseudocode for $A = [1, 2, 3, 4, 2, 5, 1]$, $n = 7$?

```
void funn(Integer A[], Integer n)
    Set seen = empty
    Integer result = -1
    for i = 0 to n-1
        if (seen.contains(A[i]))
            result = A[i]
            break
        end if
        seen.add(A[i])
    end for
    Print result
end function
funn(A, 7)
```

- a. 1
- b. 2
- c. 5
- d. 4

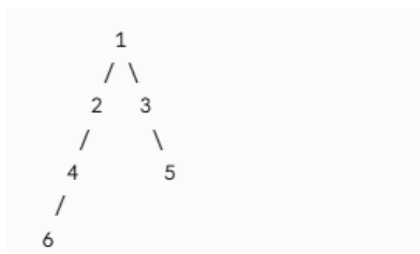
Answer: 2

Explanation:

i=0: $A[0]=1 \rightarrow$ not seen $\rightarrow$ add 1 $\rightarrow$ seen={1}
i=1: $A[1]=2 \rightarrow$ not seen $\rightarrow$ add 2 $\rightarrow$ seen={1,2}
i=2: $A[2]=3 \rightarrow$ not seen $\rightarrow$ add 3 $\rightarrow$ seen={1,2,3}
i=3: $A[3]=4 \rightarrow$ not seen $\rightarrow$ add 4 $\rightarrow$ seen={1,2,3,4}
i=4: $A[4]=2 \rightarrow$ already in seen $\rightarrow$ result=2 $\rightarrow$ break
Print result $\rightarrow$ 2



32. What will be the output of the following pseudocode for the binary tree below?



```
Integer tree(Node root)
    if (root == NULL)
        return 0
    end if
    Integer leftH = tree(root.left)
```

```

Integer rightH = tree(root.right)
return 1 + max(leftH, rightH)
end function
Print tree(root)

```

- a. 5
- b. 2
- c. 4
- d. 3

Answer: 4

Explanation:

```

tree(6) = 1 + max(tree(NULL), tree(NULL)) = 1 + max(0,0) = 1
tree(4) = 1 + max(tree(6), tree(NULL)) = 1 + max(1,0) = 2
tree(2) = 1 + max(tree(4), tree(NULL)) = 1 + max(2,0) = 3
tree(5) = 1 + max(tree(NULL), tree(NULL)) = 1
tree(3) = 1 + max(tree(NULL), tree(5)) = 1 + max(0,1) = 2
tree(1) = 1 + max(tree(2), tree(3)) = 1 + max(3,2) = 4
Print tree(root) → 4

```

33. What will be the DFS traversal output starting from node 0 for the following adjacency list?



```

0: [1, 2]
1: [0, 3]
2: [0, 3]
3: [1, 2]

```

```

void dsa(Integer node, Set visited)
    if (node in visited)
        return
    end if
    visited.add(node)
    Print node
    for each neighbor in adjList[node]
        dsa(neighbor, visited)
    end for
end function
dsa(0, empty Set)

```

- a. 0 1 3 2
- b. 0 1 2 3
- c. 0 2 3 1
- d. 0 3 1 2

Answer: 0 1 3 2

Explanation:

```
dsa(0): not visited → visit → Print 0 → visited={0}
Explore neighbors of 0: [1, 2] → first go to 1
dsa(1): not visited → visit → Print 1 → visited={0,1}
Explore neighbors of 1: [0, 3] → 0 already visited → go to 3
dsa(3): not visited → visit → Print 3 → visited={0,1,3}

Explore neighbors of 3: [1, 2] → 1 visited → go to 2
dsa(2): not visited → visit → Print 2 → visited={0,1,2,3}
Explore neighbors of 2: [0, 3] → both visited → return
Unwind back – all nodes processed
Output order: 0 1 3 2
```

34. What will be the output of the following pseudocode for coins = [1, 3, 4], amount = 6?

```
Integer coins(Integer coins[], Integer amount)
Integer dp[amount+1]
dp[0] = 0
for i = 1 to amount
    dp[i] = infinity
    for each c in coins
        if (c <= i and dp[i-c] + 1 < dp[i])
            dp[i] = dp[i-c] + 1
        end if
    end for
end for
return dp[amount]
end function
Print coins([1,3,4], 6)
```

- a. 6
- b. 4
- c. 3
- d. 2

Answer: 2

Explanation:

```
dp[0]=0
dp[1]: min coin usable=1 → dp[1]=dp[0]+1=1
dp[2]: coin=1 → dp[2]=dp[1]+1=2
dp[3]: coin=1→dp[2]+1=3; coin=3→dp[0]+1=1 → dp[3]=1 (min)
dp[4]: coin=1→dp[3]+1=2; coin=3→dp[1]+1=2; coin=4→dp[0]+1=1 → dp[4]=1 (min)
dp[5]: coin=1→dp[4]+1=2; coin=3→dp[2]+1=3; coin=4→dp[1]+1=2 → dp[5]=2 (min)
dp[6]: coin=1→dp[5]+1=3; coin=3→dp[3]+1=2; coin=4→dp[2]+1=3 → dp[6]=2 (min)
return dp[6] → 2 (using coins 3+3=6)
```

35. What will be the output of the following pseudocode for `expr = "{[()]}"`?

```
Boolean funn(String expr)
  Stack S = empty
  for each ch in expr
    if (ch == '{' or ch == '[' or ch == '(')
      push(S, ch)
    else
      if (S is empty)
        return false
      end if
      Character top = pop(S)
      if (not matches(top, ch))
        return false
      end if
    end if
  end for
  return S is empty
end function
Print funn("{[()]}")
```

- a. error
- b. false
- c. true
- d. empty

Answer: true

Explanation:



```
ch='{' → opening → push → Stack: [{]
ch='[' → opening → push → Stack: [{, []
ch='(' → opening → push → Stack: [{, [, (]
ch=')' → closing → pop '(' → matches ')' → Stack: [{, []
ch=']' → closing → pop '[' → matches ']' → Stack: [{]
ch='}' → closing → pop '{' → matches '}' → Stack: []
End of string → Stack is empty → return true
Print funn("{[()]}") → true
```

36. What will be the output of the following pseudocode?

```
void processTasks()
  Queue Q = empty
  enqueue(Q, 3)
  enqueue(Q, 1)
  enqueue(Q, 4)
  Integer sum = 0
  while (Q is not empty)
    sum = sum + dequeue(Q)
  end while
  Print sum
end function
```

- a. 4
- b. 0
- c. 7
- d. 8

Answer: 8

Explanation:

```
enqueue(3) → Queue: [3]
enqueue(1) → Queue: [3, 1]
enqueue(4) → Queue: [3, 1, 4]
dequeue() → removes 3 → sum = 0+3 = 3 → Queue: [1, 4]
dequeue() → removes 1 → sum = 3+1 = 4 → Queue: [4]
dequeue() → removes 4 → sum = 4+4 = 8 → Queue: []
Queue empty → loop ends
Print sum → 8
```

37. What will be the output of the following pseudocode for A = [12, 35, 1, 10, 34, 1], n = 6?

```
Integer numbers(Integer A[], Integer n)
    Integer first = -infinity, second = -infinity
    for i = 0 to n-1
        if (A[i] > first)
            second = first
            first = A[i]
        else if (A[i] > second and A[i] != first)
            second = A[i]
        end if
    end for
    return second
end function
Print numbers(A, 6)
```

- a. 10
- b. 12
- c. 34
- d. 35

Answer: 34

Explanation:

```
i=0: A[0]=12 → 12>first(-inf) → second=-inf, first=12
i=1: A[1]=35 → 35>first(12) → second=12, first=35
i=2: A[2]=1 → 1>first? no; 1>second(12)? no → no change
i=3: A[3]=10 → 10>first? no; 10>second(12)? no → no change
i=4: A[4]=34 → 34>first(35)? no; 34>second(12) and 34≠35 → second=34
i=5: A[5]=1 → no changes
return second → 34
```

38. What will be the output of the following pseudocode for S1 = "LISTEN", S2 = "SILENT"?

```
Boolean funn(String S1, String S2)
  if (length(S1) != length(S2))
    return false
  end if
  Integer count[26] = {0}
  for i = 0 to length(S1)-1
    count[S1[i] - 'A'] = count[S1[i] - 'A'] + 1
    count[S2[i] - 'A'] = count[S2[i] - 'A'] - 1
  end for
  for i = 0 to 25
    if (count[i] != 0)
      return false
    end if
  end for
  return true
end function
Print funn("LISTEN", "SILENT")
```

- a. false
- b. true
- c. error
- d. empty

Answer: true

Explanation:



Lengths equal (6 = 6) → proceed

For each matching pair of characters, increment count for S1's char, decrement for S2's char

Since "LISTEN" and "SILENT" contain exactly the same letters (L,I,S,T,E,N), each increment is canceled by a corresponding decrement

After processing all characters, every entry in count[] returns to 0

Final loop checks all counts are 0 → true

Print funn(...) → true

39. What will be the output of the following pseudocode for x = 20?

```
Integer x = 20
Integer y = x >> 2
Integer z = x << 1
Print y + z
```

- a. 5
- b. 40
- c. 25

d. 45

Answer: 45

Explanation:

```
x = 20 → binary: 10100
x >> 2 (right shift by 2, divides by 4) → 10100 >> 2 = 00101 = 5 → y = 5
x << 1 (left shift by 1, multiplies by 2) → 10100 << 1 = 101000 = 40 → z = 40
y + z = 5 + 40 = 45
Print y + z → 45
```

40. What will be the output of the following pseudocode?

```
Integer calc(Integer n, Integer acc)
    if (n == 0)
        return acc
    end if
    return calc(n - 1, acc * n)
end function
Print calc(5, 1)
```

a. 45

b. 120

c. 24

d. 60



Answer: 120

Explanation:

```
calc(5, 1) → n≠0 → call calc(4, 1*5=5)
calc(4, 5) → call calc(3, 5*4=20)
calc(3, 20) → call calc(2, 20*3=60)
calc(2, 60) → call calc(1, 60*2=120)
calc(1, 120) → call calc(0, 120*1=120)
calc(0, 120) → n==0 → return acc → return 120
Print calc(5,1) → 120
```

41. What will be the output of the following pseudocode for matrix A (2x3)?

```
A =
1 2 3
4 5 6
```

```

void any(Integer A[][], Integer rows, Integer cols)
    Integer B[cols][rows]
    for i = 0 to rows-1
        for j = 0 to cols-1
            B[j][i] = A[i][j]
        end for
    end for
    printMatrix(B)
end function
any(A, 2, 3)

```

- a. 1 4 / 2 5 / 3 6
- b. 1 2 3 / 4 5 6
- c. 4 5 6 / 1 2 3
- d. 3 2 1 / 6 5 4

Answer: 1 4 / 2 5 / 3 6

Explanation:

A is 2 rows x 3 cols; B will be 3 rows x 2 cols
i=0,j=0: B[0][0]=A[0][0]=1
i=0,j=1: B[1][0]=A[0][1]=2
i=0,j=2: B[2][0]=A[0][2]=3
i=1,j=0: B[0][1]=A[1][0]=4
i=1,j=1: B[1][1]=A[1][1]=5
i=1,j=2: B[2][1]=A[1][2]=6
B = [[1,4],[2,5],[3,6]]
printMatrix(B) → 1 4 / 2 5 / 3 6



42. What will be the output of the following pseudocode?

```

Integer n = 1
Integer sum = 0
do
    sum = sum + n
    n = n + 2
while (n <= 9)
Print sum

```

- a. 16
- b. 30
- c. 20
- d. 25

Answer: 25

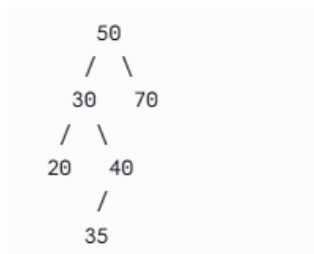
Explanation:

```

n=1: sum = 0+1 = 1; n = 1+2 = 3
Check n<=9 → 3<=9 → true → continue
n=3: sum = 1+3 = 4; n = 3+2 = 5
Check n<=9 → 5<=9 → true → continue
n=5: sum = 4+5 = 9; n = 5+2 = 7
Check n<=9 → 7<=9 → true → continue
n=7: sum = 9+7 = 16; n = 7+2 = 9
Check n<=9 → 9<=9 → true → continue
n=9: sum = 16+9 = 25; n = 9+2 = 11
Check n<=9 → 11<=9 → false → loop ends
Print sum → 25

```

43. What will be the output of the following pseudocode for the BST below, searching for key = 45?



(assume searching 45, which does not exist)



KN ACADEMY

```

Boolean funn(Node root, Integer key)
    if (root == NULL)
        return false
    end if
    if (root.value == key)
        return true
    else if (key < root.value)
        return funn(root.left, key)
    else
        return funn(root.right, key)
    end if
end function
Print funn(root, 45)

```

- a. false
- b. true
- c. error
- d. NULL

Answer: false

Explanation:

```

funn(50, 45): 45 ≠ 50; 45 < 50 → go left → funn(30, 45)
funn(30, 45): 45 ≠ 30; 45 > 30 → go right → funn(40, 45)
funn(40, 45): 45 ≠ 40; 45 > 40 → go right → funn(NULL, 45) [40 has no right child]
funn(NULL, 45): root == NULL → return false
Result propagates back up → final return → false
Print funn(root, 45) → false

```

44. What will be the output of the following pseudocode?

```

Integer calc(Integer n)
    if (n == 0)
        return 0
    end if
    return (n mod 10) + calc(n / 10)
end function
Print calc(1234)

```

- a. 9
- b. 12
- c. 10
- d. 1234

Answer: 10

Explanation:



```

calc(1234) = (1234 mod 10) + calc(1234/10) = 4 + calc(123)
calc(123) = (123 mod 10) + calc(12) = 3 + calc(12)
calc(12) = (12 mod 10) + calc(1) = 2 + calc(1)
calc(1) = (1 mod 10) + calc(0) = 1 + 0 = 1
Unwind: calc(12) = 2 + 1 = 3
calc(123) = 3 + 3 = 6
calc(1234) = 4 + 6 = 10
Print calc(1234) → 10

```

45. What will be the output of the following pseudocode for a = 6, b = 9?

```

Integer a = 6
Integer b = 9
a = a ^ b
b = a ^ b
a = a ^ b
Print a
Print b

```

- a. 9 6
- b. 6 9

c. 0 15

d. 15 0

Answer: 9 6

Explanation:

Initial: a=6 (0110), b=9 (1001)
Step 1: $a = a \oplus b = 0110 \oplus 1001 = 1111$ (15) $\rightarrow a=15$
Step 2: $b = a \oplus b = 1111 \oplus 1001 = 0110$ (6) $\rightarrow b=6$ (original a value)
Step 3: $a = a \oplus b = 1111 \oplus 0110 = 1001$ (9) $\rightarrow a=9$ (original b value)
Final values: a=9, b=6 (values swapped without a temp variable)
Print a $\rightarrow$ 9
Print b $\rightarrow$ 6

46. What will be the output of the following pseudocode for $A = [3, 7, 2, 7, 5, 7, 1]$, $n = 7$?

```
void Freq(Integer A[], Integer n)
    Integer maxVal = A[0]
    for i = 1 to n-1
        if (A[i] > maxVal)
            maxVal = A[i]
        end if
    end for
    Integer count = 0
    for i = 0 to n-1
        if (A[i] == maxVal)
            count = count + 1
        end if
    end for
    Print maxVal
    Print count
end function
Freq(A, 7)
```

a. 7,2

b. 7,3

c. 5,1

d. 7,4

Answer: 7,3

Explanation:

First loop finds maxVal: compare 3,7,2,7,5,7,1 $\rightarrow$ maximum is 7

Second loop counts occurrences of 7 in the array

$A = [3, 7, 2, 7, 5, 7, 1] \rightarrow 7$ appears at index 1, 3, and 5 $\rightarrow$ count = 3

Print maxVal $\rightarrow$ 7

Print count → 3

47. What will be the output of the following pseudocode for S = "PROGRAMMING"?

```
void funn(String S)
    Set seen = empty
    String result = ""
    for i = 0 to length(S)-1
        if (not seen.contains(S[i]))
            result = result + S[i]
            seen.add(S[i])
        end if
    end for
    Print result
end function
funn("PROGRAMMING")
```

- a. PROGAMING
- b. PROGRAMING
- c. PROGAMIN
- d. PROGRAMMIN

Answer: PROGAMIN

Explanation:



```
S = "PROGRAMMING"
i=0: 'P' → not seen → result="P" → seen={P}
i=1: 'R' → not seen → result="PR" → seen={P,R}
i=2: 'O' → not seen → result="PRO" → seen={P,R,O}
i=3: 'G' → not seen → result="PROG" → seen={P,R,O,G}
i=4: 'R' → seen → skip
i=5: 'A' → not seen → result="PROGA" → seen={P,R,O,G,A}
i=6: 'M' → not seen → result="PROGAM" → seen={P,R,O,G,A,M}
i=7: 'M' → seen → skip
i=8: 'I' → not seen → result="PROGAMI" → seen updated
i=9: 'N' → not seen → result="PROGAMIN"
i=10: 'G' → seen → skip
Final result → "PROGAMIN"
```

48. What will be the output of the following pseudocode for A = [4, 5, 2, 10, 8], n = 5 (find next greater for last element only)?

```
Integer nextGreater(Integer A[], Integer n)
    Stack S = empty
    Integer result = -1
```

```

for i = 0 to n-1
    while (S is not empty and top(S) < A[i])
        pop(S)
    end while
    push(S, A[i])
end for
return top(S)
end function
Print nextGreater(A, 5)

```

- a. 10
- b. 8
- c. 2
- d. 6

Answer: 8

Explanation:

```

i=0: A[0]=4 → stack empty → push → Stack: [4]
i=1: A[1]=5 → top(4) < 5 → pop 4 → Stack: [] → push 5 → Stack: [5]
i=2: A[2]=2 → top(5) < 2? no → push 2 → Stack: [5, 2]
i=3: A[3]=10 → top(2) < 10 → pop 2 → Stack: [5]; top(5) < 10 → pop 5 → Stack: [] → push
10 → Stack: [10]
i=4: A[4]=8 → top(10) < 8? no → push 8 → Stack: [10, 8]
Final top(S) → 8
Print nextGreater(A,5) → 8

```

49. What will be the output of the following pseudocode for Queue Q = [1, 2, 3, 4]?

```

void funn(Queue Q)
    Stack S = empty
    while (Q is not empty)
        push(S, dequeue(Q))
    end while
    while (S is not empty)
        enqueue(Q, pop(S))
    end while
    printQueue(Q)
end function
funn(Q)

```

- a. 1 2 3 4
- b. 1 3 2 4
- c. 4 1 3 2
- d. 4 3 2 1

Answer: 4 3 2 1

Explanation:

Q = [1,2,3,4] (front to rear)
Dequeue all into stack: dequeue 1→push→Stack:[1]; dequeue 2→push→Stack:[1,2];
dequeue 3→push→Stack:[1,2,3]; dequeue 4→push→Stack:[1,2,3,4] (4 on top)
Now Q is empty, Stack (bottom to top): [1,2,3,4]
Pop from stack and enqueue back: pop 4→enqueue→Q:[4]; pop 3→enqueue→Q:[4,3];
pop 2→enqueue→Q:[4,3,2]; pop 1→enqueue→Q:[4,3,2,1]
Final Queue (front to rear): [4,3,2,1]
printQueue(Q) → 4 3 2 1

50. What will be the output of the following pseudocode for the binary tree below?



```
Integer funn(Node root)
    if (root == NULL)
        return 0
    end if
    return 1 + funn(root.left) + funn(root.right)
end function
Print funn(root)
```

- a. 6
- b. 5
- c. 7
- d. 4

Answer: 6

Explanation:

```
funn(4) = 1 + funn(NULL) + funn(NULL) = 1 + 0 + 0 = 1
funn(5) = 1 + 0 + 0 = 1
funn(6) = 1 + 0 + 0 = 1
funn(2) = 1 + funn(4) + funn(5) = 1 + 1 + 1 = 3
funn(3) = 1 + funn(NULL) + funn(6) = 1 + 0 + 1 = 2
funn(1) = 1 + funn(2) + funn(3) = 1 + 3 + 2 = 6
Print funn(root) → 6
```

51. What will be the output of the following pseudocode for adjacency list (directed graph)?

```
0: [1]
1: [2]
2: [0]
```

```
Boolean hasCycleUtil(Integer node, Set visited, Set recStack, Map adjList)
    visited.add(node)
    recStack.add(node)
    for each neighbor in adjList[node]
        if (neighbor not in visited)
            if (hasCycleUtil(neighbor, visited, recStack, adjList))
                return true
            end if
        else if (neighbor in recStack)
            return true
        end if
    end for
    recStack.remove(node)
    return false
end function
Print hasCycleUtil(0, empty, empty, adjList)
```

- a. true
- b. false
- c. error
- d. 1



Answer: true

Explanation:

1. hasCycleUtil(0): visited={0}, recStack={0}; neighbor 1 not visited → recurse hasCycleUtil(1,...)
2. hasCycleUtil(1): visited={0,1}, recStack={0,1}; neighbor 2 not visited → recurse hasCycleUtil(2,...)
3. hasCycleUtil(2): visited={0,1,2}, recStack={0,1,2}; neighbor 0 is visited AND 0 is in recStack → return true
4. Propagates true back through hasCycleUtil(1) and hasCycleUtil(0)
5. Print hasCycleUtil(0,...) → true (cycle 0→1→2→0 detected)

52. What will be the output of the following pseudocode for S1 = "ABC", S2 = "AC"?

```
Integer func(String S1, String S2)
    Integer m = length(S1), n = length(S2)
    Integer dp[m+1][n+1]
    for i = 0 to m
        for j = 0 to n
            if (i == 0 or j == 0)
                dp[i][j] = 0
```

```

        else if (S1[i-1] == S2[j-1])
            dp[i][j] = dp[i-1][j-1] + 1
        else
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])
        end if
    end for
end for
return dp[m][n]
end function
Print func("ABC", "AC")

```

- a. 0
- b. 1
- c. 2
- d. 3

Answer: 2

Explanation:

```

S1="ABC" (m=3), S2="AC" (n=2)
dp[0][] and dp[][0] = 0 (base cases)
dp[1][1]: S1[0]='A', S2[0]='A' → match → dp[1][1]=dp[0][0]+1=1
dp[1][2]: S1[0]='A', S2[1]='C' → no match → dp[1][2]=max(dp[0][2], dp[1][1])=max(0,1)=1

dp[2][1]: S1[1]='B', S2[0]='A' → no match → dp[2][1]=max(dp[1][1], dp[2][0])=max(1,0)=1
dp[2][2]: S1[1]='B', S2[1]='C' → no match → dp[2][2]=max(dp[1][2], dp[2][1])=max(1,1)=1

dp[3][1]: S1[2]='C', S2[0]='A' → no match → dp[3][1]=max(dp[2][1], dp[3][0])=max(1,0)=1
dp[3][2]: S1[2]='C', S2[1]='C' → match → dp[3][2]=dp[2][1]+1=1+1=2
return dp[3][2] → 2

```

53. What will be the output of the following pseudocode?

```

Integer calc(Integer base, Integer exp)
    if (exp == 0)
        return 1
    end if
    if (exp mod 2 == 0)
        Integer half = calc(base, exp / 2)
        return half * half
    else
        return base * calc(base, exp - 1)
    end if
end function
Print calc(2, 5)

```

- a. 25
- b. 10

c. 16

d. 32

Answer: 32

Explanation:

```
calc(2,5): exp=5 is odd → return 2 * calc(2,4)
calc(2,4): exp=4 is even → half = calc(2,2); return half * half
calc(2,2): exp=2 is even → half = calc(2,1); return half * half
calc(2,1): exp=1 is odd → return 2 * calc(2,0)
calc(2,0): exp=0 → return 1
```

Unwind: $\text{calc}(2,1) = 2 * 1 = 2$

```
calc(2,2) = half * half = 2 * 2 = 4
calc(2,4) = half * half = 4 * 4 = 16
calc(2,5) = 2 * 16 = 32
Print calc(2,5) → 32
```

54. What will be the output of the following pseudocode for $A = [0, 1, 0, 3, 12]$, $n = 5$?

```
void sample(Integer A[], Integer n)
    Integer j = 0
    for i = 0 to n-1
        if (A[i] != 0)
            swap(A[i], A[j])
            j = j + 1
        end if
    end for
    Print A
end function
sample(A, 5)
```

a. 1 0 3 12 0

b. 1 3 12 0 0

c. 0 0 1 3 12

d. 1 3 12 0 12

Answer: 1 3 12 0 0

Explanation:

1. Initial: $A=[0,1,0,3,12]$, $j=0$
2. $i=0$: $A[0]=0 \rightarrow$ skip (condition false)
3. $i=1$: $A[1]=1 \rightarrow$ non-zero $\rightarrow$ swap($A[1], A[0]$) $\rightarrow A=[1,0,0,3,12]$; $j=1$
4. $i=2$: $A[2]=0 \rightarrow$ skip
5. $i=3$: $A[3]=3 \rightarrow$ non-zero $\rightarrow$ swap($A[3], A[1]$) $\rightarrow A=[1,3,0,0,12]$; $j=2$

6. i=4: A[4]=12 → non-zero → swap(A[4],A[2]) → A=[1,3,12,0,0]; j=3
7. Loop ends
8. Print A → 1 3 12 0 0

55. What will be the output of the following pseudocode for S = "AAABBCCCC"?

```
void funn(String S)
    String result = ""
    Integer count = 1
    for i = 1 to length(S) - 1
        if (S[i] == S[i-1])
            count = count + 1
        else
            result = result + S[i-1] + toString(count)
            count = 1
        end if
    end for
    result = result + S[length(S)-1] + toString(count)
    Print result
end function
funn("AAABBCCCC")
```

- a. A3B3C4
- b. A2B2C4
- c. A3B2C3
- d. A3B2C4



Answer: A3B2C4

Explanation:

```
S = "AAABBCCCC", count=1, result=""
i=1: S[1]='A'==S[0]='A' → count=2
i=2: S[2]='A'==S[1]='A' → count=3
i=3: S[3]='B'!=S[2]='A' → result="A3"; count=1
i=4: S[4]='B'==S[3]='B' → count=2
i=5: S[5]='C'!=S[4]='B' → result="A3B2"; count=1
i=6: S[6]='C'==S[5]='C' → count=2
i=7: S[7]='C'==S[6]='C' → count=3
i=8: S[8]='C'==S[7]='C' → count=4 (last index, loop ends after this since i goes to length-1=8)
After loop: result = result + S[8] + count = "A3B2" + "C" + "4" = "A3B2C4"
Print result → A3B2C4
```

56. What will be the output of the following pseudocode for $A = [4, 1, 2, 1, 2]$, $n = 5$?

```
Integer find(Integer A[], Integer n)
    Integer result = 0
    for i = 0 to n-1
        result = result ^ A[i]
    end for
    return result
end function
Print find(A, 5)
```

- a. 4
- b. 1
- c. 2
- d. 0

Answer: 4

Explanation:

```
result = 0 initially
result = 0 ^ 4 = 4
result = 4 ^ 1 = 5 (0100 ^ 0001 = 0101)
result = 5 ^ 2 = 7 (0101 ^ 0010 = 0111)

result = 7 ^ 1 = 6 (0111 ^ 0001 = 0110)
result = 6 ^ 2 = 4 (0110 ^ 0010 = 0100)
All paired elements (1,1 and 2,2) cancel out via XOR, leaving only the single element
Print find(A,5) → 4
```

57. What will be the output of the following pseudocode?

```
Integer n = 4
Real sum = 0.0
for i = 1 to n
    sum = sum + (1.0 / i)
end for
Print sum
```

- a. 2.512
- b. 1.567
- c. 2.083
- d. 3.0

Answer: 2.083

Explanation:

1. i=1: sum = 0 + 1/1 = 1.0
2. i=2: sum = 1.0 + 1/2 = 1.5
3. i=3: sum = 1.5 + 1/3 = 1.833
4. i=4: sum = 1.833 + 1/4 = 2.083
5. Loop ends
6. Print sum → 2.083 (approximately)

58. What will be the output of the following pseudocode for a = 8, b = 12?

```
Integer calc(Integer a, Integer b)
    if (b == 0)
        return a
    end if
    return calc(b, a mod b)
end function
Print calc(8, 12)
```

- a. 4
- b. 8
- c. 12
- d. 2

Answer: 4

Explanation:

```
calc(8,12): b=12≠0 → return calc(12, 8 mod 12) = calc(12, 8)
calc(12,8): b=8≠0 → return calc(8, 12 mod 8) = calc(8, 4)
calc(8,4): b=4≠0 → return calc(4, 8 mod 4) = calc(4, 0)
calc(4,0): b==0 → return a → return 4
Unwinds back: calc(8,4)=4; calc(12,8)=4; calc(8,12)=4
Print calc(8,12) → 4
```

59. What will be the output of the following pseudocode for A = [2, 7, 11, 15], n = 4, target = 18?

```
void funn(Integer A[], Integer n, Integer target)
    Map seen = empty
    for i = 0 to n-1
        Integer need = target - A[i]
        if (seen.contains(need))
            Print need
```

```

        Print A[i]
        return
    end if
    seen[A[i]] = true
end for
end function
funn(A, 4, 18)

```

- a. 11,7
- b. 2,15
- c. 7,11
- d. 15,2

Answer: 7,11

Explanation:

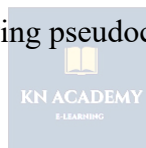
1. $i=0$: $A[0]=2 \rightarrow \text{need}=18-2=16 \rightarrow$ seen doesn't contain 16 $\rightarrow$ seen={2:true}
2. $i=1$: $A[1]=7 \rightarrow \text{need}=18-7=11 \rightarrow$ seen doesn't contain 11 $\rightarrow$ seen={2:true, 7:true}
3. $i=2$: $A[2]=11 \rightarrow \text{need}=18-11=7 \rightarrow$ seen contains 7! $\rightarrow$ Print 7, Print 11 $\rightarrow$ return
4. Output: 7 11 (need value printed first, then current element)

60. What will be the output of the following pseudocode?

```

Integer a = 10
a += 5
a -= 3
a *= 2
a /= 4
Print a

```



- a. 8
- b. 7
- c. 5
- d. 6

Answer: 6

Explanation:

1. $a=10$
2. $a += 5 \rightarrow a=10+5=15$
3. $a -= 3 \rightarrow a=15-3=12$
4. $a *= 2 \rightarrow a=12*2=24$
5. $a /= 4 \rightarrow a=24/4=6$
6. Print a $\rightarrow 6$

Part-3

61. What will be the output of the following pseudocode for A = [1, 2, 3, 4, 5], n = 5?

```
void funn(Integer A[], Integer n)
    Integer start = 0, end = n - 1
    while (start < end)
        swap(A[start], A[end])
        start = start + 1
        end = end - 1
    end while
    Print A
end function
funn(A, 5)
```

- a. 1 5 3 2 4
- b. 5 4 3 1 2
- c. 1 2 3 4 5
- d. 5 4 3 2 1

Answer: 5 4 3 2 1

Explanation:

1. Initial: A=[1,2,3,4,5], start=0, end=4
2. swap(A[0],A[4]) → A=[5,2,3,4,1]; start=1, end=3
3. Check start<end → 1<3 true → swap(A[1],A[3]) → A=[5,4,3,2,1]; start=2, end=2
4. Check start<end → 2<2 false → loop ends
5. Print A → 5 4 3 2 1

62. What will be the output of the following pseudocode for S = "GOOD IS BETTER"?

```
void reverse(String S)
    Array words = split(S, " ")
    Integer n = length(words)
    String result = ""
    for i = n-1 to 0 step -1
        result = result + words[i]
        if (i != 0)
            result = result + " "
        end if
    end for
    Print result
end function
reverse("GOOD IS BETTER")
```

- a. BETTER IS GOOD

- b. GOOD IS BETTER
- c. BETTER GOOD IS
- d. IS BETTER GOOD

Answer: BETTER IS GOOD

Explanation:

```
1. words = ["GOOD", "IS", "BETTER"], n=3
2. i=2: result = "BETTER"; i≠0 → result = "BETTER "
3. i=1: result = "BETTER IS"; i≠0 → result = "BETTER IS "
4. i=0: result = "BETTER IS BETTER"... wait, recompute: result = "BETTER IS " +
   words[0]="GOOD" → "BETTER IS GOOD"; i==0 → no trailing space
5. Final result → "BETTER IS GOOD"
```

63. What will be the output of the following pseudocode for $n = 12$?

```
Integer funn(Integer n)
    return n & (n - 1)
end function
Print funn(12)
```



- a. 10
- b. 4
- c. 8
- d. 14

Answer: 8

Explanation:

```
n = 12 → binary: 1100
n - 1 = 11 → binary: 1011
n & (n-1) → 1100 AND 1011 = 1000 (bit by bit AND)
1000 in decimal = 8
Print funn(12) → 8
```

64. What will be the output (top to bottom) of the following pseudocode for Stack $S = [3, 1, 4, 2]$ (bottom to top)?

```
void insert(Stack S, Integer x)
    if (S is empty or x > top(S))
        push(S, x)
    return
end if
```

```

    Integer temp = pop(S)
    insert(S, x)
    push(S, temp)
end function

void funn(Stack S)
    if (S is not empty)
        Integer x = pop(S)
        funn(S)
        insert(S, x)
    end if
end function
funn(S)

```

a. 1 2 3 4

b. 4 3 2 1

c. 2 4 1 3

d. 3 1 4 2

Answer: 4 3 2 1

Explanation:

This recursively sorts the stack such that after funn, elements are arranged with the largest on top.

funn pops all elements recursively, then uses insert to reinsert each in sorted order (ascending from bottom to top, but insert places larger x on top when $x > \text{top}(S)$).

Given $S = [3, 1, 4, 2]$ (bottom to top), the algorithm sorts to have the smallest at the bottom and the largest at the top:

bottom $\rightarrow$ top = 1, 2, 3, 4

Printing top to bottom (as typically stacks are displayed/popped) gives:

4 3 2 1

Output (top to bottom): 4 3 2 1

65. What will be the output of the following pseudocode for Queue $Q = [1, 2, 3, 4, 5, 6]$?

```

void interleave(Queue Q, Integer n)
    Stack S = empty
    for i = 1 to n/2
        push(S, dequeue(Q))
    end for
    while (S is not empty)
        enqueue(Q, pop(S))
    end while
    for i = 1 to n/2
        enqueue(Q, dequeue(Q))
    end for

```

```

for i = 1 to n/2
    enqueue(Q, pop(S))
end for
printQueue(Q)
end function

```

- a. Depends on implementation details (ambiguous/error - S is empty in second pop loop)
- b. 1 4 2 5 3 6
- c. 4 5 6 1 2 3
- d. 1 2 3 4 5 6

Answer: option a

Explanation:

1. This pseudocode has a logical flaw: S is emptied in the "while (S is not empty)" loop (lines 6-8), so by the time line 13 tries pop(S) again, S is already empty
2. Trace: dequeue 1,2,3 → push to S → S=[1,2,3] (3 on top); Q=[4,5,6]
3. while loop: pop 3→enqueue→Q=[4,5,6,3]; pop 2→enqueue→Q=[4,5,6,3,2]; pop 1→enqueue→Q=[4,5,6,3,2,1]; now S is empty
4. Next for loop (line 9-11): dequeue+enqueue 3 times: dequeue 4→enqueue→Q=[5,6,3,2,1,4]; dequeue 5→enqueue→Q=[6,3,2,1,4,5]; dequeue 6→enqueue→Q=[3,2,1,4,5,6]
5. Final for loop (line 12-14): pop(S) — but S is now empty → this would cause an error/undefined behavior
6. This demonstrates a common pitfall: always verify a stack/queue isn't already exhausted before further pop/dequeue calls

66. What will be the output of the following pseudocode for the binary tree below?



```

Integer func(Node root)
    if (root == NULL)
        return 0
    end if
    return root.value + func(root.left) + func(root.right)
end function
Print func(root)

```

- a. 50
- b. 60

c. 45

d. 39

Answer: 60

Explanation:

```
func(3) = 3 + 0 + 0 = 3
func(7) = 7 + 0 + 0 = 7
func(20) = 20 + 0 + 0 = 20

func(5) = 5 + func(3) + func(7) = 5 + 3 + 7 = 15
func(15) = 15 + func(NULL) + func(20) = 15 + 0 + 20 = 35
func(10) = 10 + func(5) + func(15) = 10 + 15 + 35 = 60

Print func(root) → 60
```

67. What will be the output of the following pseudocode for adjacency list?

```
0: [1]
1: [0]
2: [3]
3: [2]
4: []
```



```
Integer countComponents(Integer n, Map adjList)
    Set visited = empty
    Integer count = 0
    for node = 0 to n-1
        if (node not in visited)
            count = count + 1
            DFSMark(node, visited, adjList)
        end if
    end for
    return count
end function
Print countComponents(5, adjList)
```

a. 3

b. 2

c. 4

d. 5

Answer: 3

Explanation:

1. node=0: not visited → count=1 → DFSMark visits 0 and 1 (connected) → visited={0,1}
2. node=1: already visited → skip
3. node=2: not visited → count=2 → DFSMark visits 2 and 3 (connected) → visited={0,1,2,3}
4. node=3: already visited → skip
5. node=4: not visited → count=3 → DFSMark visits only 4 (isolated node) → visited={0,1,2,3,4}
6. return count → 3
7. Print countComponents(5, adjList) → 3 (components: {0,1}, {2,3}, {4})

68. What will be the output of the following pseudocode for weights=[1,3,4], values=[15,20,30], capacity=4?

Integer funn(Integer wt[], Integer val[], Integer n, Integer cap)

Integer dp[n+1][cap+1]

for i = 0 to n

for w = 0 to cap

if (i == 0 or w == 0)

dp[i][w] = 0

else if (wt[i-1] <= w)

dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w])

else

dp[i][w] = dp[i-1][w]

end if

end for

end for

return dp[n][cap]

end function

Print funn([1,3,4], [15,20,30], 3, 4)

a. 35

b. 30

c. 45



d. 50

Answer: 35

Explanation:

```
Items: (wt=1,val=15), (wt=3,val=20), (wt=4,val=30); capacity=4

Option 1: take item3 alone (wt=4) → value=30
Option 2: take item1+item2 (wt=1+3=4) → value=15+20=35
Option 3: take item1 alone (wt=1) → value=15 (leaves capacity 3 unused efficiently, worse than above)

Best combination within capacity 4: item1+item2 = weight 4, value 35 (better than just item3's 30)

DP table computation confirms max value achievable = 35

Print funn([1,3,4], [15,20,30], 3, 4) → 35
```

69. What will be the output of the following pseudocode for $n = 4$?

```
Integer countMoves(Integer n)
    if (n == 0)
        return 0
    end if
    return 2 * countMoves(n-1) + 1
end function
Print countMoves(4)
```

a. 13

b. 14

c. 15

d. 16

Answer: 15

Explanation:

1. $\text{countMoves}(0) = 0$ (base case)
2. $\text{countMoves}(1) = 2\text{countMoves}(0) + 1 = 2 \cdot 0 + 1 = 1$
3. $\text{countMoves}(2) = 2\text{countMoves}(1) + 1 = 2 \cdot 1 + 1 = 3$
4. $\text{countMoves}(3) = 2\text{countMoves}(2) + 1 = 2 \cdot 3 + 1 = 7$
5. $\text{countMoves}(4) = 2\text{countMoves}(3) + 1 = 2 \cdot 7 + 1 = 15$
6. Print $\text{countMoves}(4) \rightarrow 15$ (matches formula $2^n - 1 = 2^4 - 1 = 15$)

70. What will be the output of the following pseudocode for $A = [1, 2, 4, 5, 6]$, $n = 5$ (numbers should be 1 to 6)?

```
Integer find(Integer A[], Integer n)

    Integer total = (n+1) * (n+2) / 2

    Integer sum = 0

    for i = 0 to n-1

        sum = sum + A[i]

    end for

    return total - sum

end function

Print find(A, 5)
```

- a. 2
- b. 3
- c. 4
- d. 5

Answer: 3

Explanation:



```
n = 5, expected numbers are 1 to (n+1) = 6

total = (n+1)(n+2)/2 = 6*7/2 = 21 (sum of numbers from 1 to 6)

sum of array A = 1 + 2 + 4 + 5 + 6 = 18

find = total - sum = 21 - 18 = 3

Print find(A,5) → 3 (the missing number in the sequence 1 to 6 is 3)
```

71. What will be the output of the following pseudocode for $S1 = "ABCD"$, $S2 = "CDAB"$?

```
Boolean isRotation(String S1, String S2)

    if (length(S1) != length(S2))
        return false
    end if

    String combined = S1 + S1
    if (combined contains S2)
        return true
    end if
    return false
```

```
end function
Print isRotation("ABCD", "CDAB")
```

- a. false
- b. true
- c. error
- d. 1

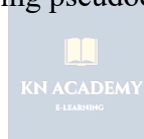
Answer: true

Explanation:

S1="ABCD", S2="CDAB", both length 4 → proceed
combined = S1 + S1 = "ABCDABCD"
Check if combined contains S2="CDAB" as a substring
Looking at "ABCDABCD": starting at index 2 → "CDAB" → found!
Since S2 is found within combined, return true
Print isRotation(...) → true

72. What will be the output of the following pseudocode?

```
Integer a = -7
Integer b = 3
Integer result = a mod b
Print result
```



- a. -1
- b. 1
- c. 2
- d. -2

Answer: -1

Explanation:

1. a = -7, b = 3
2. In most pseudocode/C-like conventions, mod follows the sign of the dividend (a)
3. $-7 / 3 = -2$ (truncated towards zero) → $-2 * 3 = -6$
4. remainder = a - (quotient * b) = $-7 - (-6) = -1$
5. result = -1
6. Print result → -1

73. What will be the output of the following pseudocode for $A = [7, 10, 4, 3, 20, 15]$, $n = 6$, $k = 3$?

```
Integer funn(Integer A[], Integer n, Integer k)
    sortDescending(A, n)
    return A[k-1]
end function
Print funn(A, 6, 3)
```

- a. 15
- b. 10
- c. 7
- d. 20

Answer: 10

Explanation:

```
A = [7, 10, 4, 3, 20, 15]
sortDescending → A = [20, 15, 10, 7, 4, 3]
k=3 → return A[k-1] = A[2]
A[2] = 10 (0-indexed: A[0]=20, A[1]=15, A[2]=10)
Print funn(A,6,3) → 10
```

74. What will be the output of the following pseudocode for $n = 10$?

```
Integer funn(Integer n)
    Integer sum = 0
    for i = 1 to n
        if (i mod 2 == 0)
            sum = sum + i
        end if
    end for
    return sum
end function
Print funn(10)
```

- a. 19
- b. 20
- c. 25
- d. 30

Answer: 30

Explanation:

Even numbers from 1 to 10: 2, 4, 6, 8, 10

i=2: sum=0+2=2

i=4: sum=2+4=6

i=6: sum=6+6=12

i=8: sum=12+8=20

i=10: sum=20+10=30

Print funn(10) → 30

75. What will be the output of the following pseudocode?

```
Integer counter = 0
Integer incrementCounter()
    counter = counter + 1
    return counter
end function
Print incrementCounter()
Print incrementCounter()
Print incrementCounter()
```



- a. 1 2 3
- b. 1 1 1
- c. 0 1 2
- d. 3 3 3

Answer: 1 2 3

Explanation:

1. counter starts at 0 (global/shared variable across calls)
2. Call 1: counter = 0+1 = 1 → return 1 → Print 1
3. Call 2: counter = 1+1 = 2 → return 2 → Print 2
4. Call 3: counter = 2+1 = 3 → return 3 → Print 3
5. Output sequence: 1 2 3

76. What will be the output of the following pseudocode for n = 10, pos = 1?

```
Integer toggleBit(Integer n, Integer pos)
    return n ^ (1 << pos)
```

```
end function
Print toggleBit(10, 1)
```

- a. 11
- b. 12
- c. 8
- d. 9

Answer: 8

Explanation:

1. $n = 10 \rightarrow$ binary: 1010
2. $1 \ll \text{pos} \rightarrow 1 \ll 1 = 0010$ (binary 2)
3. $n \wedge (1 \ll \text{pos}) \rightarrow 1010 \wedge 0010 = 1000$ (bit at position 1 gets toggled: was 1, becomes 0)
4. 1000 in decimal = 8
5. Print toggleBit(10,1) $\rightarrow$ 8

77. What will be the output of the following pseudocode?

```
      1
     /\
    2  2
   /\ /\
  3 4 4 3
```



```
Boolean funn(Node t1, Node t2)
    if (t1 == NULL and t2 == NULL)
        return true
    end if
    if (t1 == NULL or t2 == NULL)
        return false
    end if

    if (t1.value != t2.value)
        return false
    end if
    return funn(t1.left, t2.right) and funn(t1.right, t2.left)
end function
Print funn(root.left, root.right)
```

- a. error
- b. false
- c. true
- d. 0

Answer: true

Explanation:

```
funn(2[left subtree], 2[right subtree]): values match (2==2) → check funn(3,3) and funn(4,4)
funn(3,3): both leaves, values match, children are NULL,NULL → returns true
funn(4,4): both leaves, values match, children are NULL,NULL → returns true
Since both sub-calls return true → funn(2,2) returns true
Print funn(root.left, root.right) → true (tree is symmetric)
```

78. What will be the output of the following pseudocode for the weighted adjacency matrix (n=3)?

```
A =
0 4 0
4 0 7
0 7 0
```

```
Integer totalWeight(Integer A[][], Integer n)
    Integer sum = 0
    for i = 0 to n-1
        for j = i+1 to n-1
            sum = sum + A[i][j]
        end for
    end for
    return sum
end function
Print totalWeight(A, 3)
```

- a. 14
- b. 7
- c. 22
- d. 11

Answer: 11

Explanation:

```
Only upper triangle (j > i) is summed to avoid double-counting symmetric weighted edges
i=0: j=1 → A[0][1]=4 → sum=4; j=2 → A[0][2]=0 → sum=4
i=1: j=2 → A[1][2]=7 → sum=4+7=11
i=2: no j>2 within range
return sum → 11
Print totalWeight(A,3) → 11
```

79. What will be the output of the following pseudocode for grid (2x2)?

```
grid =  
1 3  
1 5
```

```
Integer minPathSum(Integer grid[][], Integer rows, Integer cols)  
    Integer dp[rows][cols]  
    dp[0][0] = grid[0][0]  
    for j = 1 to cols-1  
        dp[0][j] = dp[0][j-1] + grid[0][j]  
    end for  
    for i = 1 to rows-1  
        dp[i][0] = dp[i-1][0] + grid[i][0]  
    end for  
    for i = 1 to rows-1  
        for j = 1 to cols-1  
            dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j]  
        end for  
    end for  
    return dp[rows-1][cols-1]  
end function  
Print minPathSum(grid, 2, 2)
```

- a. 5
- b. 9
- c. 7
- d. 10



Answer: 7

Explanation:

1. $dp[0][0] = grid[0][0] = 1$
2. $dp[0][1] = dp[0][0] + grid[0][1] = 1 + 3 = 4$ (first row, can only come from left)
3. $dp[1][0] = dp[0][0] + grid[1][0] = 1 + 1 = 2$ (first column, can only come from above)
4. $dp[1][1] = \min(dp[0][1], dp[1][0]) + grid[1][1] = \min(4, 2) + 5 = 2 + 5 = 7$
5. return $dp[1][1] \rightarrow 7$
6. Print $\text{minPathSum}(\text{grid}, 2, 2) \rightarrow 7$

80. What will be the output of the following pseudocode for $A = [2, 4, 6, 8]$, $n = 4$?

```
Integer func(Integer A[], Integer n)  
    if (n == 0)  
        return 0  
    end if  
    return A[n-1] + func(A, n-1)  
end function  
Print func(A, 4)
```

- a. 20
- b. 18
- c. 16
- d. 24

Answer: 20

Explanation:

```
func(A,4) = A[3] + func(A,3) = 8 + func(A,3)
func(A,3) = A[2] + func(A,2) = 6 + func(A,2)
func(A,2) = A[1] + func(A,1) = 4 + func(A,1)
func(A,1) = A[0] + func(A,0) = 2 + func(A,0)
func(A,0) = 0 (base case)

Unwind: func(A,1) = 2 + 0 = 2
func(A,2) = 4 + 2 = 6
func(A,3) = 6 + 6 = 12
func(A,4) = 8 + 12 = 20

Print func(A,4) → 20
```

E-LEARNING

81. What will be the output of the following pseudocode for A = [16, 17, 4, 3, 5, 2], n = 6?

```
void funn(Integer A[], Integer n)
    Integer maxRight = A[n-1]
    Print maxRight
    for i = n-2 to 0 step -1
        if (A[i] > maxRight)
            Print A[i]
            maxRight = A[i]
        end if
    end for
end function
funn(A, 6)
```

- a. 17 5 2
- b. 2 5 17
- c. 2 5 17 16
- d. 16 17 5 2

Answer: 2 5 17

Explanation:

1. maxRight = A[5] = 2 → Print 2
2. i=4: A[4]=5 → 5 > 2 → Print 5; maxRight=5
3. i=3: A[3]=3 → 3 > 5? no → skip
4. i=2: A[2]=4 → 4 > 5? no → skip
5. i=1: A[1]=17 → 17 > 5 → Print 17; maxRight=17
6. i=0: A[0]=16 → 16 > 17? no → skip
7. Output order (as printed): 2 5 17

82. What will be the output of the following pseudocode for S = "SWISS"?

```
Character sample(String S)
    Integer count[26] = {0}
    for i = 0 to length(S)-1
        count[S[i] - 'A'] = count[S[i] - 'A'] + 1
    end for
    for i = 0 to length(S)-1
        if (count[S[i] - 'A'] == 1)
            return S[i]
        end if
    end for
    return '_'
end function
Print sample("SWISS")
```

- a. I
- b. S
- c. W
- d. _



Answer: W

Explanation:

```
S = "S W I S S", count each letter: S appears 3 times, W appears 1 time, I appears 1 time
First loop builds frequency: count['S']=3, count['W']=1, count['I']=1
Second loop scans left to right:
i=0 → S[0]='S' → count=3 → not 1 → skip
i=1 → S[1]='W' → count=1 → return 'W' immediately
Print sample("SWISS") → W
```

83. What will be the output of the following pseudocode for a = 10, b = 20?

```
Integer countBitsToFlip(Integer a, Integer b)
```

```

Integer x = a ^ b
Integer count = 0
while (x > 0)
    count = count + (x mod 2)
    x = x / 2
end while
return count
end function
Print countBitsToFlip(10, 20)

```

- a. 2
- b. 5
- c. 3
- d. 4

Answer: 4

Explanation:

1. $a=10 \rightarrow$ binary: 01010; $b=20 \rightarrow$ binary: 10100
2. $x = a \wedge b = 01010 \wedge 10100 = 11110$
3. Count set bits in x (11110):
4. $x=30(11110)$: $30 \bmod 2=0$, count=0, $x=15$
5. $x=15(01111)$: $15 \bmod 2=1$, count=1, $x=7$
6. $x=7(00111)$: $7 \bmod 2=1$, count=2, $x=3$
7. $x=3(00011)$: $3 \bmod 2=1$, count=3, $x=1$
8. $x=1(00001)$: $1 \bmod 2=1$, count=4, $x=0$
9. Loop ends $\rightarrow$ return count $\rightarrow 4$
10. Print countBitsToFlip(10,20) $\rightarrow 4$

KNACADEMY

84. What will be the output of the following pseudocode?

```

void minStackDemo()
    Stack S = empty
    Stack minS = empty
    push(S, 5); push(minS, 5)
    push(S, 2); push(minS, 2)
    push(S, 8); push(minS, 2)
    push(S, 1); push(minS, 1)
    pop(S); pop(minS)
    Print top(minS)
end function

```

- a. 1
- b. 2
- c. 5

d. 8

Answer: 2

Explanation:

1. push(5) → S:[5], minS:[5] (min so far=5)
2. push(2) → S:[5,2], minS:[5,2] (min so far=2, since 2<5)
3. push(8) → S:[5,2,8], minS:[5,2,2] (min so far stays 2, since 8>2)
4. push(1) → S:[5,2,8,1], minS:[5,2,2,1] (min so far=1, since 1<2)
5. pop(S) removes 1 from S; pop(minS) removes 1 from minS → S:[5,2,8], minS:[5,2,2]
6. top(minS) → 2
7. Print top(minS) → 2

85. What will be the first 3 outputs of the following pseudocode?

```
void generateBinary(Integer n)
    Queue Q = empty
    enqueue(Q, "1")
    for i = 1 to n
        String front = dequeue(Q)
        Print front
        enqueue(Q, front + "0")
        enqueue(Q, front + "1")
    end for
end function
generateBinary(3)
```



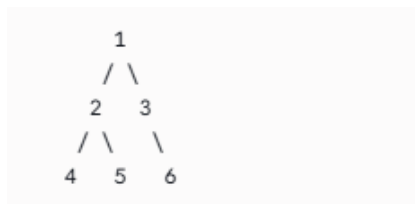
- a. 1, 10, 11
- b. 1, 11, 10
- c. 10, 11, 1
- d. 1, 10, 100

Answer: 1, 10, 11

Explanation:

1. Initial: enqueue "1" → Queue: ["1"]
2. i=1: dequeue "1" → Print "1"; enqueue "10", enqueue "11" → Queue: ["10","11"]
3. i=2: dequeue "10" → Print "10"; enqueue "100", enqueue "101" → Queue: ["11","100","101"]
4. i=3: dequeue "11" → Print "11"; enqueue "110", enqueue "111" → Queue: ["100","101","110","111"]
5. Loop ends (n=3 iterations done)
6. Output sequence: 1, 10, 11

86. What will be the output of the following pseudocode for the binary tree below?



```
void funn(Node root)
    Queue Q = empty
    enqueue(Q, root)
    while (Q is not empty)
        Node node = dequeue(Q)
        Print node.value
        if (node.left != NULL)
            enqueue(Q, node.left)
        end if
        if (node.right != NULL)
            enqueue(Q, node.right)
        end if
    end while
end function
funn(root)
```

- a. 1 3 2 6 5 4
- b. 4 5 2 6 3 1
- c. 1 2 3 4 5 6
- d. 1 2 4 5 3 6



Answer: 1 2 3 4 5 6

Explanation:

```
enqueue(1) → Queue: [1]
dequeue 1 → Print 1; enqueue 2, enqueue 3 → Queue: [2,3]
dequeue 2 → Print 2; enqueue 4, enqueue 5 → Queue: [3,4,5]
dequeue 3 → Print 3; enqueue 6 (no left child) → Queue: [4,5,6]
dequeue 4 → Print 4; no children → Queue: [5,6]
dequeue 5 → Print 5; no children → Queue: [6]
dequeue 6 → Print 6; no children → Queue: []
Output order: 1 2 3 4 5 6
```

87. What will be the output of the following pseudocode for $A = [2, 3, -2, 4]$, $n = 4$?

```
Integer funn(Integer A[], Integer n)
    Integer maxProd = A[0], minProd = A[0], result = A[0]
    for i = 1 to n-1
        if (A[i] < 0)
            swap(maxProd, minProd)
```

```

        end if
        maxProd = max(A[i], maxProd * A[i])
        minProd = min(A[i], minProd * A[i])
        result = max(result, maxProd)
    end for
    return result
end function
Print funn(A, 4)

```

- a. 24
- b. 6
- c. -2
- d. 4

Answer: 6

Explanation:

```

maxProd=2, minProd=2, result=2

i=1: A[1]=3 (positive, no swap); maxProd=max(3,2*3=6)=6; minProd=min(3,2*3=6)=3; result=max(2,6)=6

i=2: A[2]=-2 (negative → swap maxProd,minProd) → maxProd=3, minProd=6; then maxProd=max(-2,3*-2=-6)=-2; minProd=min(-2,6*-2=-12)=-12; result=max(6,-2)=6

i=3: A[3]=4 (positive, no swap); maxProd=max(4,-2*4=-8)=4; minProd=min(4,-12*4=-48)=-48; result=max(6,4)=6

return result → 6

Print funn(A,4) → 6

```

88. What will be the output of the following pseudocode for A = [1, 3, 5, 7, 9, 11], target = 7?

```

Integer search(Integer A[], Integer low, Integer high, Integer target)
    if (low > high)
        return -1
    end if
    Integer mid = (low + high) / 2
    if (A[mid] == target)
        return mid
    else if (A[mid] < target)
        return search(A, mid+1, high, target)
    else
        return search(A, low, mid-1, target)
    end if
end function
Print search(A, 0, 5, 7)

```

- a. 4
- b. 2
- c. 3

d. -1

Answer: 3

Explanation:

```
search(A,0,5,7): mid=(0+5)/2=2 → A[2]=5 → 5<7 → recurse search(A,3,5,7)
search(A,3,5,7): mid=(3+5)/2=4 → A[4]=9 → 9>7 → recurse search(A,3,3,7)
search(A,3,3,7): mid=(3+3)/2=3 → A[3]=7 → matches! → return 3
Print search(A,0,5,7) → 3
```

89. What will be the output of the following pseudocode for $A = [1, 2, 3, 4]$, $n = 4$?

```
void productExceptSelf(Integer A[], Integer n)
    Integer result[n]
    result[0] = 1
    for i = 1 to n-1
        result[i] = result[i-1] * A[i-1]
    end for
    Integer rightProduct = 1
    for i = n-1 to 0 step -1
        result[i] = result[i] * rightProduct
        rightProduct = rightProduct * A[i]
    end for
    Print result
end function
productExceptSelf(A, 4)
```

a. 1 2 3 4

b. 24 12 8 6

c. 6 8 12 24

d. 4 3 2 1

Answer: 24 12 8 6

Explanation:

1. First loop (prefix products): $result = [1, \dots]$; $i=1$: $result[1]=result[0]A[0]=11=1$; $i=2$: $result[2]=result[1]A[1]=12=2$; $i=3$: $result[3]=result[2]A[2]=23=6 \rightarrow result = [1, 1, 2, 6]$
2. Second loop (suffix products, right to left): $rightProduct=1$
3. $i=3$: $result[3]=result[3]rightProduct=61=6$; $rightProduct=rightProductA[3]=14=4$
4. $i=2$: $result[2]=result[2]rightProduct=24=8$; $rightProduct=4A[2]=43=12$
5. $i=1$: $result[1]=result[1]rightProduct=112=12$; $rightProduct=12A[1]=122=24$
6. $i=0$: $result[0]=result[0]rightProduct=124=24$; $rightProduct=24A[0]=241=24$
7. Final result = [24, 12, 8, 6]
8. Print result $\rightarrow 24\ 12\ 8\ 6$

90. What will be the output (length) of the following pseudocode for S = "ABCABCBB"?

```
Integer funn(String S)
  Set seen = empty
  Integer left = 0, maxLen = 0
  for right = 0 to length(S)-1
    while (seen.contains(S[right]))
      seen.remove(S[left])
      left = left + 1
    end while
    seen.add(S[right])
    maxLen = max(maxLen, right - left + 1)
  end for
  return maxLen
end function
Print funn("ABCABCBB")
```

- a. 4
- b. 3
- c. 5
- d. 2

Answer: 3

Explanation:



S = "A B C A B C B B", using sliding window with left, right pointers

right=0('A'): not in seen → add → seen={A}; maxLen=max(0,0-0+1)=1

right=1('B'): not in seen → add → seen={A,B}; maxLen=max(1,1-0+1)=2

right=2('C'): not in seen → add → seen={A,B,C}; maxLen=max(2,2-0+1)=3

right=3('A'): in seen → remove S[0]='A', left=1 → seen={B,C}; add 'A' → seen={B,C,A}; maxLen=max(3,3-1+1)=3

right=4('B'): in seen → remove S[1]='B', left=2 → seen={C,A}; add 'B' → seen={C,A,B}; maxLen=max(3,4-2+1)=3

right=5('C'): in seen → remove S[2]='C', left=3 → seen={A,B}; add 'C' → seen={A,B,C}; maxLen=max(3,5-3+1)=3

right=6('B'): in seen → remove S[3]='A', left=4 → seen={B,C}; still contains 'B' → remove S[4]='B', left=5 → seen={C}; add 'B' → seen={C,B}; maxLen=max(3,6-5+1)=3

right=7('B'): in seen → remove S[5]='C', left=6 → seen={B}; still contains 'B' → remove S[6]='B', left=7 → seen={}; add 'B' → seen={B}; maxLen=max(3,7-7+1)=3

Final maxLen = 3

Print funn("ABCABCBB") → 3

91. What will be the output of the following pseudocode?

```
Integer x = 5
Boolean checkCondition()
  x = x + 10
  return true
```

```
end function
Boolean result = (false and checkCondition())
Print x
Print result
```

- a. 5, false
- b. 15, false
- c. 5, true
- d. 15, true

Answer: 5, false

Explanation:

1. $x = 5$ initially
2. Expression: `false and checkCondition()`
3. Due to short-circuit evaluation of "and", if the first operand is false, the second operand is NOT evaluated (since the overall result is already determined to be false)
4. `checkCondition()` is never called $\rightarrow$ x remains unchanged at 5
5. `result = false`
6. Print $x \rightarrow 5$
7. Print `result` $\rightarrow$ false



92. What will be the output of the following pseudocode for $n = 7$, $k = 3$?

```
Integer multiplyByPower(Integer n, Integer k)
    return n << k
end function
Print multiplyByPower(7, 3)
```

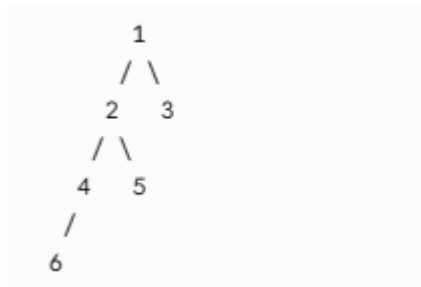
- a. 21
- b. 58
- c. 56
- d. 49

Answer: 56

Explanation:

1. $n = 7 \rightarrow$ binary: 00111
2. $n \ll k$ means shifting left by k positions, equivalent to $n * 2^k$
3. $n \ll 3 = 7 * 2^3 = 7 * 8 = 56$
4. Verify with binary: $00111 \ll 3 = 111000 = 32 + 16 + 8 = 56$
5. Print `multiplyByPower(7,3)` $\rightarrow 56$

93. What will be the output (diameter, in number of edges) of the following pseudocode for the binary tree below?



```

Integer diameter(Node root, Integer[] maxDia)
    if (root == NULL)
        return 0
    end if
    Integer leftHeight = diameter(root.left, maxDia)
    Integer rightHeight = diameter(root.right, maxDia)
    maxDia[0] = max(maxDia[0], leftHeight + rightHeight)
    return 1 + max(leftHeight, rightHeight)
end function
Integer maxDia[1] = {0}
diameter(root, maxDia)
Print maxDia[0]
  
```

- a. 4
- b. 3
- c. 5
- d. 2



Answer: 4

Explanation:

1. diameter(6): leftH=0, rightH=0 → maxDia=max(0,0)=0 → return 1+max(0,0)=1
2. diameter(4): leftH=diameter(6)=1, rightH=0 → maxDia=max(0,1+0)=1 → return 1+max(1,0)=2
3. diameter(5): leftH=0, rightH=0 → maxDia stays max(1,0)=1 → return 1
4. diameter(2): leftH=diameter(4)=2, rightH=diameter(5)=1 → maxDia=max(1,2+1)=3 → return 1+max(2,1)=3
5. diameter(3): leftH=0, rightH=0 → maxDia stays 3 → return 1
6. diameter(1): leftH=diameter(2)=3, rightH=diameter(3)=1 → maxDia=max(3,3+1)=4 → return 1+max(3,1)=4
7. Final maxDia[0] = 4 (path: 6→4→2→1→3, which has 4 edges)
8. Print maxDia[0] → 4

94. What will be one valid output of the following pseudocode (Kahn's algorithm) for adjacency list?

```
0: [1, 2]
1: [3]
2: [3]
3: []
```

```
void topoSort(Integer n, Map adjList)
    Integer inDegree[n] = {0}
    for node = 0 to n-1
        for each neighbor in adjList[node]
            inDegree[neighbor] = inDegree[neighbor] + 1
        end for
    end for
    Queue Q = empty
    for node = 0 to n-1
        if (inDegree[node] == 0)
            enqueue(Q, node)
        end if
    end for
    while (Q is not empty)
        Integer node = dequeue(Q)
        Print node
        for each neighbor in adjList[node]
            inDegree[neighbor] = inDegree[neighbor] - 1
            if (inDegree[neighbor] == 0)
                enqueue(Q, neighbor)
            end if
        end for
    end while
end function
topoSort(4, adjList)
```

a. 0 2 1 3

b. 0 1 2 3

c. 1 2 0 3

d. 3 2 1 0

Answer: 0 1 2 3

Explanation:

1. Compute in-degrees: node0=0 (no incoming), node1=1 (from 0), node2=1 (from 0), node3=2 (from 1 and 2)
2. Initial queue: only node0 has inDegree 0 → Queue:[0]
3. Dequeue 0 → Print 0; decrement neighbors 1,2 → inDegree[1]=0, inDegree[2]=0 → enqueue both → Queue:[1,2]

4. Dequeue 1 → Print 1; decrement neighbor 3 → inDegree[3]=1 (not 0 yet) → Queue:[2]
5. Dequeue 2 → Print 2; decrement neighbor 3 → inDegree[3]=0 → enqueue 3 → Queue:[3]
6. Dequeue 3 → Print 3; no neighbors → Queue:[]
7. Output order: 0 1 2 3

95. What will be the output of the following pseudocode for S1 = "CAT", S2 = "CUT"?

```
Integer editDistance(String S1, String S2)
    Integer m = length(S1), n = length(S2)
    Integer dp[m+1][n+1]
    for i = 0 to m
        for j = 0 to n
            if (i == 0)
                dp[i][j] = j
            else if (j == 0)
                dp[i][j] = i
            else if (S1[i-1] == S2[j-1])
                dp[i][j] = dp[i-1][j-1]
            else
                dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
            end if
        end for
    end for
    return dp[m][n]
end function
Print editDistance("CAT", "CUT")
```

- a. 0
- b. 3
- c. 2
- d. 1

Answer: 1

Explanation:

1. S1="CAT", S2="CUT", both length 3
2. Base cases: dp[0][j]=j, dp[i][0]=i
3. dp[1][1]: S1[0]='C'==S2[0]='C' → match → dp[1][1]=dp[0][0]=0
4. dp[1][2]: S1[0]='C', S2[1]='U' → no match → dp[1][2]=1+min(dp[0][1],dp[0][2],dp[1][1])=1+min(1,2,0)=1
5. dp[1][3]: S1[0]='C', S2[2]='T' → no match → dp[1][3]=1+min(dp[0][2],dp[0][3],dp[1][2])=1+min(2,3,1)=2
6. dp[2][1]: S1[1]='A', S2[0]='C' → no match → dp[2][1]=1+min(dp[1][0],dp[1][1],dp[2][0])=1+min(1,0,2)=1
7. dp[2][2]: S1[1]='A', S2[1]='U' → no match → dp[2][2]=1+min(dp[1][1],dp[1][2],dp[2][1])=1+min(0,1,1)=1
8. dp[2][3]: S1[1]='A', S2[2]='T' → no match → dp[2][3]=1+min(dp[1][2],dp[1][3],dp[2][2])=1+min(1,2,1)=2

9. $dp[3][1]: S1[2]='T', S2[0]='C' \rightarrow$ no match $\rightarrow dp[3][1]=1+\min(dp[2][0], dp[2][1], dp[3][0])=1+\min(2, 1, 3)=2$
10. $dp[3][2]: S1[2]='T', S2[1]='U' \rightarrow$ no match $\rightarrow dp[3][2]=1+\min(dp[2][1], dp[2][2], dp[3][1])=1+\min(1, 1, 2)=2$
11. $dp[3][3]: S1[2]='T', S2[2]='T' \rightarrow$ match $\rightarrow dp[3][3]=dp[2][2]=1$
12. return $dp[3][3] \rightarrow 1$ (only 'A' $\rightarrow$ 'U' substitution needed)
13. Print $editDistance("CAT", "CUT") \rightarrow 1$

96. What will be the output of the following pseudocode?

```
Integer countPermutations(Integer n, Integer r)
    if (r == 0)
        return 1
    end if
    return n * countPermutations(n - 1, r - 1)
end function
Print countPermutations(5, 3)
```

- a. 20
- b. 0
- c. 120
- d. 60



Answer: 60

Explanation:

1. $countPermutations(5,3) = 5 * countPermutations(4,2)$
2. $countPermutations(4,2) = 4 * countPermutations(3,1)$
3. $countPermutations(3,1) = 3 * countPermutations(2,0)$
4. $countPermutations(2,0) = 1$ (base case, $r=0$)
5. Unwind: $countPermutations(3,1) = 3*1 = 3$
6. $countPermutations(4,2) = 4*3 = 12$
7. $countPermutations(5,3) = 5*12 = 60$
8. Print $countPermutations(5,3) \rightarrow 60$ (this computes $P(5,3) = 5!/(5-3)! = 5*4*3 = 60$)

97. What will be the output of the following pseudocode for $A = [-7, 1, 5, 2, -4, 3, 0]$, $n = 7$ (equilibrium index: sum of elements left equals sum of elements right)?

```
Integer equilibriumIndex(Integer A[], Integer n)
    Integer totalSum = 0
    for i = 0 to n-1
        totalSum = totalSum + A[i]
    end for
```

```

Integer leftSum = 0
for i = 0 to n-1
    totalSum = totalSum - A[i]
    if (leftSum == totalSum)
        return i
    end if
    leftSum = leftSum + A[i]
end for
return -1
end function
Print equilibriumIndex(A, 7)

```

- a. 3
- b. 2
- c. 4
- d. -1

Answer: 3

Explanation:

1. totalSum = -7+1+5+2-4+3+0 = 0
2. i=0: totalSum = 0-(-7)=7; leftSum(0)==totalSum(7)? no → leftSum=0+(-7)=-7
3. i=1: totalSum = 7-1=6; leftSum(-7)==totalSum(6)? no → leftSum=-7+1=-6
4. i=2: totalSum = 6-5=1; leftSum(-6)==totalSum(1)? no → leftSum=-6+5=-1
5. i=3: totalSum = 1-2=-1; leftSum(-1)==totalSum(-1)? yes! → return 3
6. Print equilibriumIndex(A,7) → 3

98. What will be the output of the following pseudocode for a = -5, b = 3?

```

Boolean funn(Integer a, Integer b)
    return (a ^ b) < 0
end function
Print funn(-5, 3)

```

- a. true
- b. false
- c. error
- d. 0

Answer: true

Explanation:

a = -5 (in two's complement, sign bit is 1/negative), b = 3 (sign bit is 0/positive)

XOR of two numbers with different sign bits will have sign bit = 1 (since $1 \text{ XOR } 0 = 1$)

$a \wedge b \rightarrow$ since a is negative and b is positive, the result of XOR will be negative (sign bit set)

Check condition $(a \wedge b) < 0 \rightarrow \text{true}$, since the XOR result has its sign bit set

This confirms a and b have opposite signs

Print `funn(-5,3) → true`

99. What will be the output of the following pseudocode for $A = [1, 2, 3, -4, -1, 4]$, $n = 6$?

```
Integer sample(Integer A[], Integer n, Integer startIdx)
    for i = startIdx to n-1
        if (A[i] < 0)
            return A[i]
        end if
    end for
    return 0
end function
Print sample(A, 6, 2)
```

a. 0

b. -1

c. -4

d. 4



Answer: -4

Explanation:

```
Start scanning from index 2 (A[2]=3)
i=2: A[2]=3 → not negative → continue
i=3: A[3]=-4 → negative → return -4 immediately
Print sample(A,6,2) → -4
```

100. What will be the output of the following pseudocode?

```
Integer funn(String expr)
    Stack S = empty
    Array tokens = split(expr, " ")
    for i = length(tokens)-1 to 0 step -1
        String token = tokens[i]
        if (token is a number)
            push(S, token)
```

```

        else
            Integer a = pop(S)
            Integer b = pop(S)
            if (token == '+')
                push(S, a + b)
            else if (token == '*')
                push(S, a * b)
            end if
        end if
    end for
    return pop(S)
end function
Print funn("* + 2 3 4")

```

- a. 24
- b. 14
- c. 20
- d. 9

Answer: 20

Explanation:

```

tokens = ["*", "+", "2", "3", "4"], scanning right to left
token="4" → number → push → Stack:[4]
token="3" → number → push → Stack:[4,3]
token="2" → number → push → Stack:[4,3,2]
token="+" → pop a=2, pop b=3 → push a+b=2+3=5 → Stack:[4,5]
token="*" → pop a=5, pop b=4 → push a*b=5*4=20 → Stack:[20]
return pop(S) → 20
Print funn("* + 2 3 4") → 20

```